1. الـ Case Sensitivity (اللعب بالأحرف)

في الـ **HTTP/1.1**، كان البروتوكول "مبحبحها" شوية؛ يعني لو كتبت `Content-Type` أو `content-type` السيرفر هيفهمك. لكن زي ما ذكرت، مع ظهور **HTTP/2** و **HTTP/3**، الموضوع بقى فيه "نضافة" أكتر:

- كل الـ Headers بقت **Lowercase** (حروف صغيرة) بشكل إجباري في البروتوكول نفسه.
- **الزتونة في Node.js:** الـ Node.js ذكي جداً، هو أوتوماتيك بيحول كل الـ Headers اللي جاية لك لـ **Lowercase** عشان يريحك من قصة هي مكتوبة كابيتال ولا سمول. يعني لو الزبون بعت `User-Agent` أنت هتستلمها `req.headers['user-agent']`.

2. الـ Pseudo-headers (العناوين الوهمية)

دي بقى "افتكاسة" الـ HTTP/2 عشان يسرع الدنيا.

- في HTTP/1.x كان السيرفر بيبعت أول سطر فيه الـ Status زي `HTTP/1.1 200 OK`.
- في HTTP/2، شالوا السطر ده وحطوا بداله **Pseudo-headers** بتبدأ بـ `:` زي `:status: 200` و `:path: /`.
- دول اسمهم "وهميين" لأنهم مش Headers حقيقية المستخدم اللي حاططها، دي معلومات أساسية البروتوكول هو اللي حاططها في الأول.

1. إيه حكاية الـ prefix `X-` ؟

زمان (قبل 2012)، كان أي حد عايز يخترع Header خاص بيه (زي `X-Custom-Data` أو `X-Framework`) لازم يبدأ بـ `X-` عشان يقول للعالم: "يا جماعة ده Header تجريبي ومش رسمي".

2. ليه اتلغت (Deprecated)؟

المشكلة كانت لما الـ Header ده "ينجح" ويتحول لـ Standard.

- **المعضلة:** المتصفحات والسيرفرات كانت اتعودت خلاص على الاسم اللي فيه `X-`. لما ييجوا يحولوه لـ Standard ويشيلوا الـ `X-` (زي ما حصل مع `X-Frame-Options`) كان لازم السيرفرات تدعم الاسمين مع بعض للأبد عشان الكود القديم ما يضربش.
- **RFC 6648:** لو السيرفر مش عارفه هيتجاهله. "`X-` باسمه على طول من غير Header جت وقالت: "بطلوا عك، سموا الـ Header في المستقبل، مش هنحتاج نغير الكود Standard وخلاص، ولو بقى.

3. الـ IANA Registry (الدفاتر الرسمية)

دلوقتي أي Header "محترم" لازم يكون متسجل في سجلات **IANA**. ده "السجل المدني" بتاع الـ HTTP. لو عايز تعرف أي Header حالته إيه (Permanent, Deprecated, Obsoleted)، بترجع للـ Registry ده.

1. تصنيف الـ Headers حسب السياق (Context)

دي التقسيمة اللي هتحتاجها وأنت بتكتب كود السيرفر:

- **Request Headers:** وهل الزبون ده معاه (`User-Agent`) دي "البطاقة الشخصية" للطلب. بتعرفك المتصفح نوعه إيه تصريح دخول ولا لأ (`Authorization`).
- **Response Headers:** أو لو عايز توجهه لصفحة (`Server`) دي "رسالة السيرفر" للزبون. بتقول له السيرفر نوعه إيه تانية (`Location`).
- **Representation Headers:** JSON مثلاً: "يا متصفح، اللي باعتولك ده ملف) دي بتوصف **"المحتوى"** نفسه ومضغوط بصيغة Gzip").
- **Payload Headers:** وطريقة نقلها **"حجم البيانات"** دي بتوصف (زي `Content-Length`).

---

2. تصنيف الـ Headers حسب التعامل مع الـ Proxies

دي حتة "للمحترفين" عشان تفهم الداتا بتمشي إزاي في "مواسير" الإنترنت:

- **End-to-end Headers:** دي headers (أو" نفسها طويل". بتطلع من السيرفر وتفضل ماشية لحد ما توصل للمتصفح مهما عدت على (العكس Proxies أو Load Balancers في النص.
- **Hop-by-hop Headers:** دي headers" نفسها قصير". بتخص الوصلة الحالية بس. الـ Proxy، بياخدها، ينفذ اللي فيها **وما يبعتهاش** للي بعده.
  - *ملحوظة:* الـ `Connection` هو اللي بيحدد مين الـ headers اللي تتعامل معاملة الـ Hop-by-hop.

```
const server = http.createServer((req, res) => {
  // Representation Header: بتعرف المتصفح نوع الداتا
  res.setHeader('Content-Type', 'application/json');

  // Response Header: معلومة إضافية عن السيرفر
  res.setHeader('X-Powered-By', 'My-Node-App');

  const data = { message: "Welcome to the deep dive!" };
  res.end(JSON.stringify(data));
});
```

1. الـ Authentication (البوابة والتصريح)

دي عملية "مين أنت؟ ومعاك إيه يثبت؟"

- **Authorization:** ده "التصريح" اللي بيبعته المتصفح (Client) للسيرفر (غالباً بيبقى Token).
- **WWW-Authenticate:** بيقولك: "يا بطل، عشان (Unauthorized)، السيرفر بيرد بيه لو أنت داخل "من غير عزم تدخل هنا لازم تستخدم الطريقة الفلانية (مثلاً Bearer أو Basic)".
- **Proxy-Authenticate/Authorization:** هي اللي طالبة تعرف (Proxy) "نفس الفكرة بس لو فيه "بوابة وسيطة هويتك قبل ما تعديك للسيرفر الأصلي.

---

2. الـ Caching (توفير المجهود)

دي أهم حتة للـ Performance. بدل ما نطلب الداتا كل شوية، "نركنها" شوية.

- **Cache-Control:** ده "الريس" بتاع السيكشن ده. هو اللي بيقول: "احفظ النسخة دي عندك لمدة ساعة (`max-age=3600`) "أو "أوعى تحفظ حاجة (`no-store`).
- **ETag:** دي "بصمة" الملف. السيرفر بيدي للملف رقم فريد، لو الملف اتغير، الرقم بيتغير. دي أدق طريقة عشان نعرف الملف "هو هو" ولا لأ.
- **Expires:** الطريقة القديمة لتحديد وقت انتهاء صلاحية الملف (تاريخ محدد).

---

3. الـ Conditionals (الطلبات المشروطة)

دي قمة الذكاء في الـ HTTP. المتصفح بيقول للسيرفر: "أنا عندي نسخة قديمة، لو سمحت ابعتلي الجديد **فقط لو** حصل تغيير".

- **If-None-Match:** المتصفح بيبعت الـ **ETag** اللي عنده ويقول: "لو الملف اللي عندك بصمته مش زي اللي معايا، ابعتهولي". ومابيبعتش الداتا تاني، وده بيوفر رهيب Payload. لو طلعوا زي بعض، السيرفر بيرد بـ **304 Not Modified**.
- **Last-Modified / If-Modified-Since:** نفس الفكرة بس معتمد على "تاريخ آخر تعديل".

```
const server = http.createServer((req, res) => {
  // مثال يدوي لإرسال ETag
  const eTagValue = "v1.2.3";

  if (req.headers['if-none-match'] === eTagValue) {
    res.statusCode = 304; // "أنت عندك أحدث حاجة، مش هبعت داتا"
    return res.end();
  }

  res.setHeader('ETag', eTagValue);
  res.setHeader('Cache-Control', 'public, max-age=86400'); // احفظها يوم كامل
  res.end("<h1>Fresh Content from Server!</h1>");
});
```

1. الـ Connection Management (إدارة الوصلة)

- **Connection: keep-alive**: دي معناها "يا سيرفر ما تقفلش الخط في وشي". المتصفح بيطلب إنه يخلي الوصلة مفتوحة عشان يبعت عليها كذا طلب ورا بعض بدل ما يفتح ويقفل كل شوية، وده بيوفر وقت ومجهود كبير.

2. الـ Content Negotiation (مفاوضات المحتوى)

دي مرحلة "اتفاق" بين الطرفين:

- **Accept**: المتصفح بيقول: "أنا بفهم HTML وصور، ما تبعتليش فيديو".

- **Accept-Encoding**: "عشان توفر في الباقة؟ ممكن تبعتلي الداتا مضغوطة (gzip)".
- **Accept-Language**: "لو سمحت لو عندك نسخة عربي ابعتها".

3. الـ CORS (أمن الدولة للمواقع)

دي أهم حتة لما تبدأ تعمل API:

- المتصفح بيخاف لما موقع (أ) يطلب داتا من سيرفر (ب). الـ **Access-Control-Allow-Origin** هو اللي بيقول للمتصفح: "ماتخافش، أنا سامح للموقع ده إنه ياخد مني داتا". من غيرها، المتصفح هيقفل الطلب فوراً.

4. الـ Cookies (الذاكرة الصغيرة)

- **Set-Cookie**: السيرفر بيبعتها للمتصفح عشان يقوله "افتكر الكلمة دي".
- **Cookie**: المتصفح بيبعتها للسيرفر في كل طلب عشان يقوله "أنا فلان اللي قلت لي افتكره من شوية". دي أساس الـ Session والـ Login.

5. الـ Downloads (التحميل)

- **Content-Disposition**: ولا يطلّع لك شباك "حفظ الملف (inline) دي اللي بتخلي المتصفح يقرر: هل يفتح الملف قدامك (attachment)؟ باسم"

---

1. الـ Redirects (تحويل المسار)

دي أكتر حاجة هتستخدمها لما اليوزر يبعت Form.

- **Location**: أنت عايز (صفحة بيضاء) `/message` أنت مش عايز تسيبه في صفحة الـ Message، لما اليوزر يبعت كدة المتصفح. **302** Status Code ومعاها `res.setHeader('Location', '/')` ترجعه تاني للرئيسية. فبتبعت هيفهم إنه لازم يروح العنوان الجديد فوراً.

2. الـ Range Requests (تحميل أجزاء من الملف)

دي "الزتونة" في الـ Media Players (زي يوتيوب أو ساوند كلاود).

- بدل ما السيرفر يبعت ملف الفيديو الـ 1 جيجا كله، المتصفح بيبعت **Range** (مثلاً: "هات لي من بايت رقم 500 لحد 1000").
- السيرفر بيرد بـ **Content-Range** و **206 Partial Content** Status Code. دي بتخليك تقدر تعمل Pause و Resume للتحميل براحتك.

3. الـ Message Body Info (وصف الوجبة)

- **Content-Length**: ويطلع شريط) ده حجم الداتا بالظبط. مهم جداً عشان المتصفح يعرف هو استلم قد إيه وفاضل له قد إيه التحميل Progress Bar صح).

- **Content-Type**: ده "التاج" اللي بيعرف المتصفح هو بيتعامل مع `text/html` ولا `application/json` ولا `image/png` ..

1. الـ Transfer Coding & Message Body (نقل البيانات)

لما بتتعامل مع Node.js، إحنا مش بنبعت الداتا كلها "خبطة واحدة".

- `Content-Length` : لو أنت عارف حجم الداتا (زي ملف صغير)، بتبعت طوله بالبايت.
- `Transfer-Encoding: chunked` : ده "نجم" الـ Node.js. السيرفر بيبعت الداتا على حتت (Chunks) من غير ما يكون عارف الحجم النهائي في البداية. ده اللي بيخلي الـ **Streaming** شغال بكفاءة.

---

2. الـ WebSockets (الاتصال الحي)

الـ Headers اللي أنت ذكرتها بتلعب دور في الـ **Handshake** بتاع الـ WebSockets.

- المتصفح بيبعت Header اسمه `Upgrade: websocket` .
- السيرفر (Node.js) لو وافق، بيقلب البروتوكول من HTTP لاتصال "حي" ومفتوح طول الوقت (Full-duplex). ده اللي بنستخدمه في الـ **Chat apps** والـ **Real-time notifications**.

---

3. الـ Security & Integrity (أمن الدولة للموقع)

- `Integrity-Policy` **(SRI)**: ملمسهاش "هاكر" JavaScript زي الـ (CDN) ده بيضمن إن الملفات اللي بتنزلها من متغيرة، المتصفح مش هيشغل الكود (Hash) وغير في الكود بتاعها. لو البصمة.
- `Clear-Site-Data` : دي حركة "نضافة" من السيرفر، يقدر يطلب من المتصفح يمسح كل الـ Cookies والـ Cache بتوع الموقع لو اليوزر عمل Logout مثلاً.

---

1. الـ WebSockets (الاتصال اللي مش بيقفل)

الـ Headers اللي بتبدأ بـ `Sec-WebSocket-` هي المسؤولة عن عملية الـ **Handshake**.

- لما المتصفح يحب يفتح "شات" مثلاً، بيبعت `Upgrade: websocket` ومعاها `Sec-WebSocket-Key` .
- السيرفر (Node.js) بياخد المفتاح ده، يعمل عليه عملية حسابية معينة، ويرده في `Sec-WebSocket-Accept` . كدة الاتصال اتحول من HTTP عادي لاتصال ثنائي (Bi-directional).

---

2. الـ Transfer Coding (الـ Chunked Data)

أهم واحد هنا هو `Transfer-Encoding: chunked`.

- في Node.js، لما بنبعت بيانات كبيرة (زي ملف فيديو أو داتا تقيلة من DB)، مش بنستنى لما نجمعها كلها في الذاكرة.
- بنبعتها "حتت حتت". الـ Header ده بيقول للمتصفح: "استقبل اللي جالك دلوقتي، ولسه فيه حتت تانية جاية في الطريق". ده أساس الـ **Streams** اللي ماكس بيشرحها.

---

3. الـ Server-Timing (عينك على السيرفر)

الـ `Server-Timing` ده Header "صايع" جداً للمطورين.

- بيسمح للسيرفر إنه يبعت بيانات عن "الوقت" اللي خده في كل عملية (مثلاً: الـ Database خدت 50ms، والـ Image processing خد 100ms).
- الجميل إنك بتشوف الداتا دي مرسومة في الـ **Network Tab** في متصفحك (Chrome DevTools).

---

1. الـ Client Hints (المتصفح بيوشوش السيرفر)

دي بديلة للـ `User-Agent` القديم اللي كان بيبقى سطر طويل ومعقد.

- **الهدف:** السيرفر يقدر يبعت نسخة "مفصلة" من الموقع بناءً على إمكانيات جهازك.
- **Sec-CH-UA-Mobile:** `?1` لو (True يعني)، السيرفر يبعت نسخة الموبايل فوراً.
- **Sec-CH-Device-Memory:** لو الرامات ضعيفة، السيرفر ممكن يبعت كود JavaScript خفيف عشان الجهاز ميهنجش.
- **Sec-CH-DPR:** لو الشاشة Retina (جودتها عالية)، السيرفر يبعت صور بـ Resolution عالي.

---

2. الـ Network Client Hints (إلحقني بالخفيف)

دي عبقرية عشان الناس اللي النت عندها ضعيف:

- **Save-Data:** لو اليوزر مفعل خاصية "توفير البيانات"، السيرفر (Node.js) ممكن يقلل جودة الصور أو يلغي الفيديوهات اللي بتشتغل لوحدها أوتوماتيك.
- **ECT (Effective Connection Type):** السيرفر بيعرف لو أنت شغال على 4G ولا 2G، وبناءً عليه يقرر يبعت لك إيه.

---

3. الـ Attribution Reporting (الإعلانات من غير تجسس)

دي محاولة من المتصفحات (زي Chrome) إنهم يلغوا الـ Third-party cookies.

- بدل ما الشركة تعرف أنت رحت فين وجيت منين بالتفصيل، المتصفح نفسه هو اللي بيسجل إنك شفت إعلان وضغطت عليه، ويبعت "تقرير" للسيرفر من غير ما يكشف هويتك بالكامل.

---

4. الـ Compression Dictionary (ضغط البيانات للمحترفين)

- **Use-As-Dictionary:** يتفقوا على "قاموس" كلمات مشترك. بدل ما (Client) دي تكنولوجيا جديدة بتخلي السيرفر والعملية نبعت الكلمات الطويلة كل شوية، بنبعت "كود" صغير بيشير للكلمة في القاموس ده، وده بيقلل حجم الداتا المنقولة بنسبة مهولة.

---

1. الـ Global Privacy Control (بديل الـ DNT)

زمان كان فيه (Do Not Track) `DNT` ، بس الشركات كانت بتطنشها. دلوقتي فيه `Sec-GPC` :

- ده هيدر بيقولك: "اليوزر ده مش عايز بياناته تتباع أو تتشارك مع أطراف تالتة".
- **في كودك:** لو لقيت الهيدر ده قيمته `1` في الـ `req.headers` المفروض سيرفرك يحترم ده وما يبعتش بيانات اليوزر لشركات الإعلانات.

2. الـ Topics API (الإعلانات بالذوق)

دي محاولة جوجل لتعويض الـ Cookies. بدل ما نتتبع اليوزر هو راح فين بالظبط:

- المتصفح بيعرف "اهتمامات" اليوزر (مثلاً: مهتم بالكورة، البرمجة، الطبخ).
- `Sec-Browsing-Topics` : بيبعت لك الاهتمامات دي عشان لو عايز تعرض إعلانات "لايقة" على اليوزر من غير ما تعرف هو مين شخصياً.

3. الـ Origin-Agent-Cluster (عزل العمليات)

ده هيدر أمني تقيل:

- بيطلب من المتصفح إنه يحط الموقع بتاعك في **Process** (عملية) لوحده تماماً في الرامات، بعيد عن أي Tab تانية.
- ده بيمنع هجمات زي **Spectre** اللي ممكن تسرب بيانات من الذاكرة بين المواقع.

1. الـ Idempotency-Key (ممنوع التكرار)

دي حتة **Advanced** جداً ومهمة لو بتعمل نظام مدفوعات (Payments):

- **المشكلة:** لو اليوزر داس على زرار "ادفع" والنت قطع، فداس تاني، ممكن البنك يسحب الفلوس مرتين!
- **الحل:** الـ `Idempotency-Key` بيبعت كود فريد مع الطلب. السيرفر (Node.js) بيسجل الكود ده، ولو جاله طلب تاني بنفس الكود في وقت قصير، بيفهم إن ده تكرار ومش بينفذ العملية تاني.

2. عيلة الـ X-Forwarded (مين اللي باعت؟)

لما السيرفر بتاعك يكبر وتستخدم **Proxy** (زي Nginx) أو **Load Balancer**:

- السيرفر بتاعك هيشوف إن كل الطلبات جاية من الـ Proxy مش من اليوزر الحقيقي.
- **الزتونة:** الـ `X-Forwarded-For` هو اللي بيشيل الـ **IP الحقيقي** بتاع اليوزر اللي قاعد في بيته، عشان تقدر تعرف مكانه أو تحظره لو عمل مشكلة.

3. الـ Speculation-Rules (التنبؤ بالمستقبل)

دي تكنولوجيا بتخلي المواقع سريعة بجنون:

- السيرفر بيبعت قواعد للمتصفح بتقول له: "غالباً اليوزر هيدوس على اللينك ده كمان شوية، روحي حمليه في الخلفية دلوقتي (Prerender)".
- لما اليوزر يدوس فعلاً، الصفحة بتفتح في **0 ثانية** لأنها كانت جاهزة في الذاكرة.